PromiseTracker v3 system specification

Version 3.0, draft, 10 November 2026.

This is the full specification for PromiseTracker v3. It covers what the system does, how it is built, and the technical detail a developer needs to build it. Each section opens with a short plain-language summary, so a reader who is not a developer can follow the design and skip the detail underneath.

A shorter companion document, the product specification, explains the same system without the technical detail.

Contents

1. Overview
2. Key terms
3. How the system is put together
4. Plugins
5. Verification plugins
6. Other plugins
7. Data model
8. Publishing rules
9. The pipeline
10. The public site
11. APIs
12. Security
13. Errors and logging
14. Testing
15. Deployment and operations
16. Performance
17. Development workflow and delivery
18. Design decisions
19. Appendix A: items to confirm
20. Appendix B: what v2 does today

The word CONFIRM marks a statement that someone must check against a running system before building on it. Appendix A lists every one.

1. Overview

1.1 What the system does

PromiseTracker records what politicians and public institutions promise, checks each promise against its source, and follows what happens to it over time. Editors bring in source documents. AI proposes candidate promises, and a person reviews every one. A fact-checker verifies each promise. Editors publish it and add dated updates as events unfold. The public reads the result on a website, and partners reuse it through embeds and a data API.

1.2 What is new in v3

Version 2 cannot run without two outside services, Airtable and Meedan Check. It has no record of how a promise's status changed, and its public site does not cache, paginate or search. Appendix B lists the evidence.

Version 3 changes five things.

- **The core runs alone.** With nothing but a database and file storage, the system is a complete product. Every outside service is optional.
- **Outside services are plugins.** Verification, AI, document reading and data sources each sit behind a fixed interface, so one can be swapped for another.
- **One site serves every country.** A country is part of the web address, as in `promisetracker.org/ke`, and a field on each record.
- **Promises have a history.** Each promise carries dated updates with evidence, and its current status comes from the latest one.
- **One rule decides what is public.** Each record has a single publication setting, and hiding a parent hides everything under it.

Version 2 never launched, so there is no data to migrate.

1.3 Scope

In scope:

- Countries, actors, promises, updates, source documents, topics, statuses, citizen submissions and media
- Ingestion from uploads, links and spreadsheets, with AI-assisted extraction and human review
- Verification in the built-in review queue, in Superdesk or in Meedan Check
- A public site with country pages, profiles, promise pages, lists, search and maps
- Embeds, a read-only public API and CSV export
- English and French

Out of scope for v3. The design leaves room to add each later.

Left out	How it can be added later
Editing rights per country	Each user already has a country list that the system does not yet enforce. Enforcing it needs no change to stored data
A web address or theme per country	Routing reads the country from the path, so a domain-to-country map can sit in front of it
Public user accounts and alerts	Submissions are anonymous today, with optional contact details
Promise detection from news	A new source connector could feed it in
Verification by the crowd	Citizens send evidence. A fact-checker decides
A dedicated search engine	Search sits behind one module that can be replaced

1.4 Technology

Layer	Choice
Web framework	Next.js with the App Router, React 19, TypeScript in strict mode
Content management and admin	Payload CMS 3
Database	MongoDB
File storage	Any S3-compatible store
Document reading	Apache Tika, self-hosted
AI	Vercel AI SDK, behind the LLM provider interface
Background jobs	Payload Jobs
Error tracking	Sentry
Tests	Vitest for integration tests, Playwright for end-to-end tests

This is the same stack as v2. Version 3 adds no new infrastructure.

1.5 Rules the design must keep

1. The system runs with no outside service. With only the core plugin enabled, a team can upload documents, enter promises, verify, publish and track them. Automated tests run in this configuration.
2. Nothing reaches the public without human review.
3. Nobody types in a promise's current status. The system derives it from the latest published update.
4. The server never fetches a web address a user supplied, with one audited exception described in section 12.4.
5. One field records whether a record should be public, and one computed field records whether it is.
6. No sync, import or plugin deletes a record. A deletion upstream archives the record here.

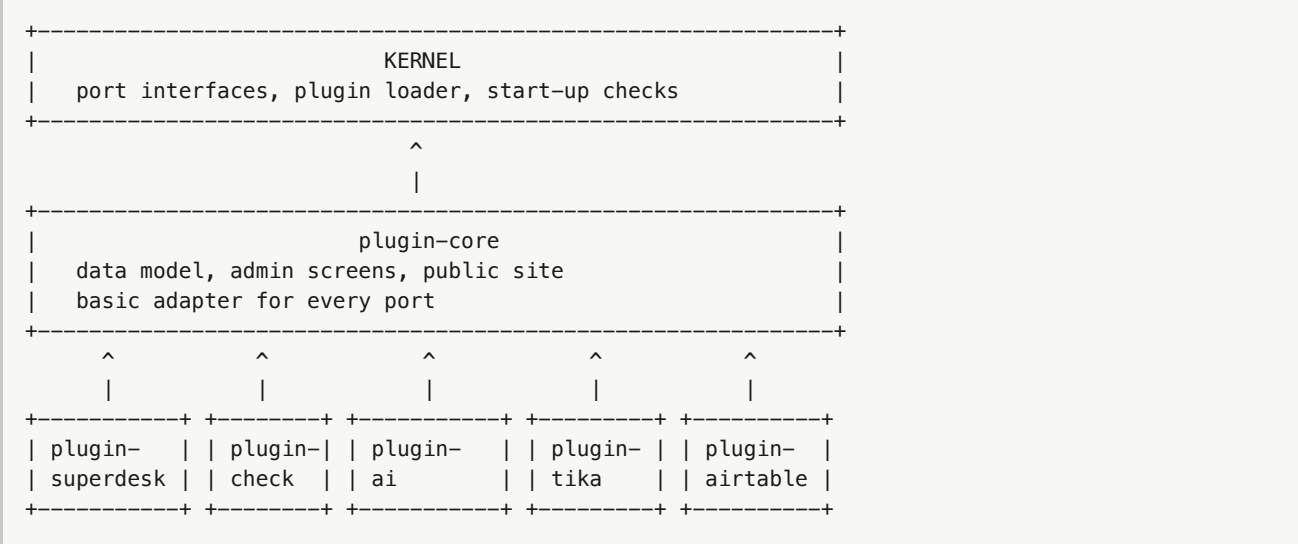
2. Key terms

Term	Meaning
Actor	Anyone or anything that makes promises: a person, a party, an institution or a place
Claim	A promise or other checkable statement. "Claim" is the name used in the data model, because the system also tracks policy pledges, budget commitments and public statements
Claim update	A dated entry in a claim's history: what happened, the status as of that date, and the evidence
Claim status	Where a claim stands, such as "in progress" or "kept". Editors define the list
Verdict	The fact-checker's finding on whether the claim is accurately quoted and attributed: unverified, verified, false or misleading
Candidate	A possible claim that AI found in a document, waiting for a person to accept or reject it
Record status	The setting an editor controls on each record: ready, hold or archived
Published status	Whether a record is live on the public site. The system computes it
Plugin	A self-contained package that adds features to the system
Port	A fixed interface that describes one job, such as "verify a claim"
Adapter	A plugin's implementation of a port. The Superdesk adapter and the Check adapter both implement the verification port
Kernel	The small base layer that defines the ports and loads the plugins
Core	The plugin that holds the whole data model and a basic adapter for every port

3. How the system is put together

In short: the system has three layers. A small kernel defines the interfaces. The core plugin holds everything PromiseTracker needs to work alone. Other plugins connect outside services and can be added or removed without changing the core.

3.1 The three layers



The kernel defines the five ports, loads plugins and checks the configuration at start-up. It knows nothing about promises or politicians.

The core plugin owns every collection in the data model, the admin screens and the public site. It also ships a basic adapter for every port, which is why the system works with no other plugin enabled.

Integration plugins each connect one outside service. Every one depends on the core, adds its own optional fields to the core's records, and provides an adapter for one or more ports. A plugin never calls another plugin directly.

3.2 Why the core is a plugin too

If the core were the application and plugins were bolted on, each integration would reach into the application's internals. That is how v2 ended up writing Airtable status messages from six different places. Making the core a plugin means it uses the same extension rules as everything else. If those rules are too weak for an integration, they are too weak for the core, and the problem shows up early.

3.3 Supported configurations

An environment variable, `PT_PLUGINS`, lists the enabled plugins. Each row below is a supported set-up.

Enabled plugins	Ingestion	Document reading	Candidates	Verification
core	Upload, link, CSV and Excel import	Plain text and HTML	None. Editors enter claims by hand	Built-in review queue

Enabled plugins	Ingestion	Document reading	Candidates	Verification
core, tika	Same	Adds PDF, Word, Excel, PowerPoint and more	None	Built-in review queue
core, tika, ai	Same	Same	AI proposes, a person reviews	Built-in review queue
core, tika, ai, superdesk	Same	Same	Same	Superdesk
core, tika, ai, check	Same	Same	Same	Meedan Check

Adding `airtable` to any row adds Airtable as a source of documents and actors.

4. Plugins

In short: a port describes a job, and an adapter does it. The system has five ports. Start-up checks make sure every required job has exactly one adapter, and ownership rules make sure two systems never write the same field.

4.1 The five ports

Port	The job	How many adapters run	Adapter in core	Other adapters
VerificationProvider	Where people verify claims	Exactly one	`native`	`superdesk` , `check`
SourceConnector	Where content comes from	Any number. Upload is always on	`upload` , `url`	`airtable` , `sheets` , `feed`
ContentExtractor	Turning a file into text	One or more, picked by file type	`plaintext`	`tika` , `ocr`
StructuredImporter	Turning spreadsheet rows into records	Any number	`csv` , `xlsx`	`sheets`
LLMProvider	Which AI model proposes candidates	Many can be installed, one is selected	None	`google` , `openai` , `anthropic` and others

Ingestion uses three ports because the inputs differ. A spreadsheet of promises that a partner compiled maps straight to records and needs no AI. A 200-page manifesto needs text extraction, then AI, then review.

4.2 What each port must provide

Every adapter also provides a `health` method that reports whether its service is reachable.

VerificationProvider

Method	Purpose
`submit(claim)`	Send a claim for verification. Returns the outside system's reference, if there is one
`pull(since)`	Fetch verdicts that changed since a saved position. Used to catch anything a push missed
`receive(payload, headers)`	Handle a push from the outside system. Optional

SourceConnector

Method	Purpose
`mode`	One of `manual` , `pull` or `push`
`list(since)`	For pull connectors, list items waiting to be brought in

Method	Purpose
`fetch(item)`	Retrieve one item's content

ContentExtractor

Method	Purpose
`accepts`	The file types this extractor handles. The most specific match wins
`extract(content)`	Return the text, a page count if known, and any metadata found

StructuredImporter

Method	Purpose
`accepts`	The file types this importer handles
`inspect(content)`	Read the column headings and sample rows without saving anything
`import(content, mapping)`	Create or update records using the column mapping

LLMProvider

Method	Purpose
`models`	The models this provider offers
`generate(request)`	Run one request and return structured output

Each method also receives a context object holding the database handle, a logger, a cancel signal and the plugin's own settings. An adapter reads its configuration from that context. It never reads environment variables itself and never reads another plugin's settings.

4.3 What a plugin declares

A plugin is a function that returns a manifest. The kernel reads the manifest and does the wiring.

Manifest entry	Purpose
`id` , `version`	Identity
`dependsOn`	Plugins that must load first
`collections` , `globals`	Record types the plugin owns outright
`extend`	Changes to record types another plugin owns, described in section 4.5
`provides`	The adapters it supplies, by port
`tasks` , `workflows`	Background jobs
`routes`	The web routes it uses, described in section 4.7
`settings`	A settings group, added under the plugin's own name
`requiredEnv`	Environment variables it needs, checked at start-up

Three environment variables select the configuration.

```
PT_PLUGINS=core,tika,ai,superdesk
PT_VERIFICATION=superdesk
PT_LLM_MODEL=google:gemini-2.5-pro
```

The system reads these at start-up. Changing a plugin needs a restart, on purpose. A plugin changes the shape of records and the list of routes, and neither can change safely while the system is serving requests.

4.4 What happens at start-up

1. Read `PT_PLUGINS` and load each manifest.
2. Check that every dependency is present and sort the plugins into load order. A circular dependency stops the start-up.
3. For each port that needs exactly one adapter, find the selected one. If there are none, or more than one, stop and name the candidates and the variable that chooses between them.
4. Combine the record types, then apply each plugin's extensions in load order.
5. Add each plugin's settings group.
6. Check the required environment variables of enabled plugins.
7. Register background jobs.
8. Log the final configuration in one line: plugins, adapters and versions.

A deployment with no way to verify claims is misconfigured, and it should fail when it is deployed, not when the first fact-checker opens a claim. Optional routes behave differently and are described in section 4.7.

4.5 Ownership rules

When a plugin takes over a piece of data, the system makes sure it is the only writer.

1. Fields that a plugin adds are always optional. A plugin may be removed after months of use, and required fields would make every existing record invalid.
2. A plugin may not change the type of a core field. It may change how the field appears and who can edit it.
3. A field that a plugin owns is read-only in the admin. Only the plugin and a super-admin can write it, and the system logs a super-admin override.
4. If a plugin owns the lifecycle of a record type, the admin hides the create and delete buttons and shows a notice saying where the data comes from.
5. A plugin's fields sit in a group named after the plugin, such as `superdesk.guid`. Two plugins cannot collide, and removing a plugin removes one group.
6. A plugin never deletes a core record. A deletion upstream sets the record to archived.

When a plugin owns a job, the matching role in the admin loses it.

Plugin enabled	What it takes over	Effect in the admin
`plugin-superdesk` or `plugin-check`	The verdict on each claim	Reviewers can no longer set a verdict in the admin. They still review AI candidates there
`plugin-airtable`	Intake of documents and actors	Contributors can no longer create those records in the admin

A verification plugin owns three fields on a claim: the verdict, the verification date and the rationale. The title, statement, topic, promisor and update history stay editable in PromiseTracker whichever plugin is active.

4.6 Rules for plugin authors

1. Add only optional fields to another plugin's record types.
2. Keep every added field inside a group named after the plugin.
3. Read configuration from the context object.
4. Put route handlers in the folder described in section 4.7, and make the enabled check the first line of each.
5. Archive, never delete.
6. Keep the `health` method cheap enough to run on every health check.
7. When the plugin is disabled, return the configuration exactly as it arrived.

4.7 Plugin routes

The web framework builds its route list from files on disk, so a plugin cannot add a route while the system runs. Plugin routes therefore live in a fixed folder, one subfolder per plugin, and each plugin lists its routes in its manifest.

```
src/app/api/plugins/
  superdesk/items/route.ts     POST  receives published items
  superdesk/assets/route.ts    GET, POST  media exchange
  check/webhook/route.ts       POST  receives report events
  airtable/webhook/route.ts    POST  receives change notices
```

The first thing each handler does is check that its plugin is enabled. If it is not, the handler returns `503` and does nothing else. Tests read the route list from the manifest, so they cover every route without a hardcoded list.

5. Verification plugins

In short: a deployment verifies claims in one of three places. Each option sends a claim out for checking and brings back the same three fields: a verdict, a date and a rationale. The claim itself never leaves PromiseTracker.

5.1 Built-in review, in plugin-core

A reviewer opens a claim in the admin and sees the statement, the source document with the quote highlighted, the promisor, and how the claim was found. They set the verdict and write a rationale. A saved view called "Awaiting verification" is the queue.

The adapter does nothing, by design. `submit` sends nothing out, `pull` returns nothing, and the verdict is an ordinary editable field.

5.2 plugin-superdesk

Superdesk is the newsroom production system from Sourcefabric. It has no built-in fact-check or claim type, so the plugin represents a claim as a normal Superdesk content item with a dedicated content profile and three custom vocabularies. The newsroom must set these up in its Superdesk before the plugin can run.

The plugin owns the verdict, verification date and rationale on each claim. It adds a read-only `superdesk` group holding the item's `guid`, version, state, link, and the time and result of the last sync.

Superdesk's three APIs

API	Address prefix	Access	Used for
Internal API	`/api`	Read and write	Creating claim items. This is the only API that can write
Production API	`/prodapi/v1`	Read only	Reading items in any state, including unpublished
Content API	Varies by installation	Read only	Reading published items

CONFIRM the Content API prefix. Installations differ: some serve `/capi/items` and others `/v1/items`. CONFIRM too that the Production API is running, because many installations leave it out.

Signing in

API	How the plugin signs in
Internal API	It posts a service account's credentials to `/api/auth_db` and receives a session token. Sessions last 240 minutes by default, and the plugin signs in again when one expires
Production API	It exchanges a client id and secret for a token at `/api/auth_server/token`. The token grants read access only
Content API	It sends a subscriber token created in Superdesk

Superdesk's token service grants read access only. Creating items therefore needs a real Superdesk user account with the `archive` privilege. Limit that account to the fact-checking desk and give it a recognisable name, such as `promisetracker-bot`, so its edits are easy to spot in Superdesk's history.

How a claim maps to a Superdesk item

PromiseTracker	Superdesk field	Notes
Claim identity	`guid`	Superdesk creates it. The plugin stores it as `superdesk.guid`
Title	`headline`	
Statement	`body_html`	The exact quote in a blockquote, with the source
Summary	`abstract`	
Claim type	`subject`, scheme `claim_type`	Custom vocabulary
Topic	`subject`, scheme `claim_category`	Custom vocabulary
Verdict	`subject`, scheme `verdict`	Custom vocabulary. CONFIRM the scheme id
Rationale	`body_html` of the published fact-check	
Claim id	`extra.promisetracker_id`	Carried so the plugin can match items back
Claim link	`extra.promisetracker_url`	
Language	`language`	
Item kind	`profile`	A content profile such as `fact_check`. CONFIRM the id

The verdict is a vocabulary in `subject` because Superdesk can search `subject` values. Values stored under `extra` are not searchable unless the installation's search mapping is changed. Anything the plugin must search goes in `subject`, and anything it only carries goes in `extra`.

CONFIRM the vocabulary ids and their codes, the content profile id, and the desk and stage ids. The plugin checks all of them at start-up and refuses to start if one is wrong.

Sending a claim to Superdesk

```
POST /api/archive
Authorization: Basic <session token>
Content-Type: application/json

{
  "type": "text",
  "profile": "<fact_check profile id>",
  "headline": "Ruto: 200 new health centres by 2027",
  "body_html": "<blockquote>exact quote</blockquote>",
  "language": "en",
  "subject": [
    { "qcode": "unverified", "scheme": "verdict" },
    { "qcode": "campaign-promise", "scheme": "claim_type" },
    { "qcode": "health", "scheme": "claim_category" }
  ],
  "extra": {
    "promisetracker_id": "6745abc",
    "promisetracker_url": "https://promisetracker.org/ke/promises/..."
  },
  "task": { "desk": "<desk id>", "stage": "<stage id>" }
}
```

Superdesk assigns the `guid` and version and sets the item's state to `draft`.

The `task` entry is required in practice. Without a desk and stage the item lands in the service account's private workspace, where no fact-checker will see it.

The plugin posts to `/api/archive` and not to Superdesk's ingest route. An item that arrives through ingest gets a new `guid` when the newsroom picks it up, which breaks the link back to the claim.

Receiving verdicts from Superdesk

The plugin uses two paths and runs both.

Push. The newsroom configures PromiseTracker as a Superdesk subscriber with an HTTP push destination. When a fact-checker publishes an item, Superdesk posts it to `/api/plugins/superdesk/items` and signs the request.

```
x-superdesk-signature: sha1=<HMAC-SHA1 of the body>
```

The handler checks the signature before it reads the body. HMAC-SHA1 is an old algorithm, but it is the one Superdesk uses, so the shared secret must be long and random. CONFIRM that HTTP push is available on the installation.

Poll. Every 15 minutes a job asks the Content API for items that changed since the last check. A push can be lost during a deploy or a network fault, and the poll catches what the push missed.

```
GET <content api prefix>/items
  ?filter={"range":{"versioncreated":{"gte":"<last seen time>"}}}
  &sort=[("versioncreated",1)]
  &max_results=200
```

The Content API has limits that cause a quiet, incomplete sync if ignored.

- Its date parameters accept whole days only, so the plugin filters on `versioncreated`.
- With no date given it returns the last 7 days only. A backfill must walk through date ranges.
- It sends no change markers, so the plugin tracks the latest `versioncreated` it has seen.
- It rejects any parameter it does not recognise.
- It returns 500 items per page by default and 1,000 at most.
- A new item can take a moment to appear in results, so each poll overlaps the previous one by 10 minutes.
- It shows a subscriber only the items published to it. If the subscriber is set up wrongly, the API returns an empty list with a success code. The plugin treats three empty polls in a row as a health warning.

What each Superdesk state means

Superdesk keeps one `guid` for an item through corrections and withdrawals, and raises the version each time. It never deletes.

Superdesk state	Effect on the claim
`draft`, `in_progress`, `submitted`	In review. No change
`published`, `corrected`	Apply the verdict and rationale, and set the verification date
`killed`, `recalled`, `takedown`, `unpublished`	Withdraw. Set the verdict to unverified, set the record to hold, and notify an editor
`spiked`	Archive the claim
`scheduled`, `being_corrected`	No change. Wait for the final state

The plugin ignores a version it has already processed and applies an item only when its version is higher than the stored one. Superdesk may deliver items out of order.

The plugin both creates items and receives them. To avoid a loop, it matches every incoming item that carries a `promisetracker_id` to the existing claim and never creates a second one. It ignores items without that id.

When Superdesk fails

Problem	What the plugin does
A profile, vocabulary, desk or stage id is wrong at start-up	Refuses to start and names the id
The session expired	Signs in again, retries once, then fails the job
A push has a bad signature	Returns `401`, logs it, and does not read the body
A push refers to an unknown item	Returns `200` so Superdesk stops retrying, and notes the item for the next poll
Superdesk is unreachable	Stops calling it for a while, queues claims locally, and reports a degraded state

When Superdesk is down, editors can still create, edit and publish claims. Only new verdicts stop arriving.

5.3 plugin-check

Meedan Check is a fact-checking workspace. Unlike Superdesk, it has a claim type and a report workflow built in, so the plugin needs no custom set-up beyond an API key and a list of statuses.

The plugin owns the verdict, verification date and rationale on each claim. It adds a read-only `check` group holding the item's id in Check, its link, the report state, and the time and result of the last sync.

The Check API

Check has one GraphQL API at `https://check-api.checkmedia.org/api/graphql`. Every request carries two headers.

Header	Value
`X-Check-Token`	The workspace API key
`X-Check-Team`	The workspace id

Both values live in the plugin's settings, encrypted.

How a claim maps to a Check item

PromiseTracker	Check field	Notes
Statement	`quote`	The exact quote. Check creates an item of type `Claim`
Title and summary	`set_claim_description`	Includes a link back to the claim in PromiseTracker
Topic, claim type, country	`set_tags`	
Starting status	`set_status`	`undetermined`
Claim identity	The returned item id	The plugin stores it as `check.itemId` and matches on it later
Item link	The returned `full_url`	Stored as `check.itemUrl` so editors can open the item

Sending a claim to Check

```
mutation createProjectMedia($input: CreateProjectMediaInput!) {
  createProjectMedia(input: $input) {
    project_media { id full_url status }
  }
}
```

```
{
  "input": {
    "media_type": "Claim",
    "quote": "We will build 200 new health centres by 2027",
    "channel": { "main": 1 },
    "set_status": "undetermined",
    "set_tags": ["health", "campaign-promise", "ke"],
    "set_claim_description": "Ruto: 200 new health centres ..."
  }
}
```

Receiving verdicts from Check

As with Superdesk, the plugin uses a push and a poll.

Push. Check sends a webhook to `/api/plugins/check/webhook` when something changes in the workspace. The request carries a shared secret in its `Authorization` header, and the handler compares it before reading the body. The plugin acts on one kind of event, a report change, which Check labels `report_design`. It acknowledges every other event with `200` and ignores it.

A report event carries the report's state. When the state is `published`, the plugin reads the item's status from Check and applies the verdict. When the state is `paused`, the fact-checker has withdrawn the report, and the plugin withdraws the verdict.

Poll. Every 15 minutes a job searches Check for items with a published report and reconciles them, 100 per page.

```
query PublishedReports($query: String!) {
  search(query: $query) {
    medias { edges { node { id full_url status report_status } } }
  }
}
```

The `query` variable is `{"report_status": ["published"]}`.

How Check statuses become verdicts

Each Check workspace defines its own status list, so the plugin's settings hold a mapping from Check status to verdict. At start-up the plugin reads the workspace's statuses and checks that every one is mapped.

Check status, default workspace	Verdict in PromiseTracker
`verified`	verified
`false`	false
`undetermined`, `in_progress`	unverified
`not_applicable`	unverified
A custom status	Whatever the mapping says. An unmapped status leaves the verdict unverified and flags the claim for an editor

Report state in Check	Effect on the claim
No report, or report not published	In review. No change
`published`	Apply the mapped verdict, copy the report text as the rationale, and set the verification date
`paused`	Withdraw. Set the verdict to unverified, set the record to hold, and notify an editor

CONFIRM the workspace id, the API key's permissions, the workspace's status list, and that the workspace can send webhooks to an outside address.

When Check fails

Problem	What the plugin does
A workspace status has no mapping at start-up	Refuses to start and names the status
Check answers `429`, too many requests	Waits for the time given in the `Retry-After` header, then retries
A webhook has a wrong secret	Returns `401`, logs it, and does not read the body
The same webhook arrives twice	Recognises it by a fingerprint of the body and returns the first answer again
A webhook refers to an unknown item	Returns `200` and notes the item for the next poll
Check is unreachable	Stops calling it for a while, queues claims locally, and reports a degraded state

Version 2 kept published promises inside Check and rebuilt them from Check's reports. Version 3 keeps the claim in PromiseTracker and takes only the verdict from Check, so a Check outage or a change in Check's data never removes a claim.

6. Other plugins

6.1 plugin-ai

This plugin finds candidate claims in documents. It provides an LLM provider adapter for each AI vendor the deployment has configured, and it owns the extraction records and the review queue screen.

A deployment selects one model, such as `google:gemini-2.5-pro`, and stores the vendor's API key in the encrypted settings. Other vendors can be installed alongside it and selected later without code changes. The provider list and credential checks come from v2, which already supports fourteen vendors.

The plugin runs a small test request at start-up and on each health check. If the account has run out of credit, the health check reports a degraded state. Without that check, the first sign would be a pile of failed documents.

With this plugin disabled, documents stop after text extraction and editors enter claims by hand.

6.2 plugin-tika

This plugin reads complex file formats. It sends a file to a self-hosted Apache Tika server and gets back the text. Tika reads PDF, Word, Excel, PowerPoint, OpenDocument, RTF, EPUB and several hundred other formats.

Requests time out after two minutes. Tika counts toward the health check only when this plugin is enabled, and the public site never depends on it.

6.3 plugin-airtable

This plugin lets a team keep feeding documents and actors in from Airtable. It is an option for teams that want it. With the plugin enabled, Airtable supplies records, and the plugin may own their fields. The work queue and the processing status stay in PromiseTracker and remain visible in the admin.

For most teams a one-off spreadsheet import, described in section 9.6, is simpler than a standing sync.

7. Data model

In short: a country contains actors and documents. Documents produce claims. Each claim belongs to an actor and a topic, and collects dated updates. Citizens send submissions, which moderators can turn into updates.

```
Country
  |-- Actor (person, party, institution or place)
  |     '-- makes Claims
  |-- Document --> Extraction --> candidates
  |                                  | accepted
  |                                  v
  '--------------------------------- Claim --> Claim updates
                                     |            ^
                                     |-- Topic    | promoted
                                     '-- Status   Submission
```

The core plugin owns every record type below unless a heading says otherwise. Each record also has creation and update timestamps, which the tables leave out. "Localized" means the field holds a value per language.

7.1 Country

A country is a section of the site and a label on records. All countries share one site, one database and one set of users.

Field	Type	Notes
`code`	Text	Two-letter ISO code in lower case, such as `ke`. Unique. Used in web addresses
`alpha3`	Text	Three-letter ISO code, for data exchange
`name`, `officialName`	Text	Localized
`flag`	Media	
`defaultLocale`	`en` or `fr`	
`locales`	List	The languages this country publishes in
`geo`	Geo group	Centre point and outline, described in section 7.11
`about`	Rich text	Localized. Shown on the country page
`branding`	Group	Accent colour and header image
`recordStatus`	ready, hold, archived	Starts as hold

Setting a country to hold or archived hides every actor, claim and document under it.

7.2 Actor

One record type covers four kinds of actor. A person, a party, a ministry and a county can all make promises, and keeping them together means one list answers "who makes promises here".

Field	Type	Notes
`country`	Country	Required
`kind`	person, party, institution, place	Required
`name`	Text	Required
`shortName`	Text	Such as "UDA" or "MoH"
`slug`	Text	The name as it appears in web addresses. Unique within a country and kind
`parent`	Actor	For nesting, such as ward inside constituency inside county
`image`	Media	
`description`	Rich text	Localized
`links`	List	Official site and social accounts
`geo`	Geo group	Section 7.11
`recordStatus`	ready, hold, archived	Starts as hold

Each kind has extra fields.

Kind	Extra fields
Person	Honorific, gender, date of birth, place of origin, and a list of offices
Party	Abbreviation, founding date, colours, leader
Institution	Type, such as ministry, agency, county government or commission. Jurisdiction, head, founding date
Place	Type, such as region, county, constituency or ward. Administrative level, population, and an external boundary code

A person's offices are a list, so the record keeps their whole career.

Office field	Notes
`title`	Such as "Governor"
`institution`	The institution they serve in
`jurisdiction`	The place they govern
`party`	Their party while in this office
`from`, `to`	Dates. `to` is empty while they hold the office
`isCurrent`	Stored so lists can filter on it quickly

A person's place of origin and the place they govern are separate fields, because they often differ.

7.3 Claim

Field	Type	Notes
`country`	Country	Required
`type`	campaign-promise, policy-pledge, budget-commitment, public-statement	Starts as campaign-promise
`title`	Text	Required. Localized. The headline
`slug`	Text	Unique within a country
`statement`	Text	Required. The exact words, never paraphrased and never translated
`summary`	Rich text	Localized. Editorial context
`promisor`	Actor	Required. Who made the claim
`coPromisors`	Actors	Such as the party alongside the candidate
`jurisdiction`	Actor of kind place	Where the claim applies, if narrower than the country
`category`	Topic	Required
`source`	Document	Empty for a claim entered by hand
`sourceLocator`	Group	Page, or timestamp for audio and video
`sourceUrl`	Text	Shown as a link. The server never fetches it
`madeOn`	Date	When the claim was made
`targetDate`	Date	The deadline the claim set for itself, if any
`language`	`en` or `fr`	
`currentStatus`	Claim status	Read-only. Taken from the latest published update
`statusAsOf`	Date	Read-only
`updateCount`	Number	Read-only
`verification.verdict`	unverified, verified, false, misleading	Starts as unverified
`verification.verifiedAt`	Date	
`verification.verifiedBy`	User or text	A user, or an outside reviewer's name
`verification.rationale`	Rich text	
`verification.report`	Media	A published fact-check
`extraction`	Extraction	Set when AI found the claim
`confidence`	Number, 0 to 1	From the AI

Field	Type	Notes
`enteredBy`	ai, manual, import	
`recordStatus`	ready, hold, archived	Starts as hold

The system keeps every past version of a claim, so an editor can see what a page said on any date and can prepare a correction without taking the page down.

7.4 Claim status

Editors define the list of statuses in the admin.

Field	Type	Notes
`slug`	Text	Unique and stable, such as `kept`
`label`	Text	Required. Localized
`description`	Text	Required. Localized. Explains the status to the public
`group`	not-started, in-progress, kept, broken, inconclusive	Required
`color`, `textColor`	Hex colour	For chips and charts
`order`	Number	Display order

Every scorecard, chart and filter reads `group`. A team can define as many statuses as it wants, and each one falls into one of the five groups. Statuses are shared across countries, so countries can be compared.

7.5 Claim update

Field	Type	Notes
`claim`	Claim	Required
`date`	Date	Required. When the event happened, which may be long before it was recorded
`status`	Claim status	Required. The status as of that date
`title`	Text	Localized
`narrative`	Rich text	Required. Localized. What happened and how we know
`evidence`	List	At least one item when the status changes
`author`	User	
`attribution`	Text	Credit for a citizen, with their consent
`sourceType`	editorial, citizen-submission, import	
`submission`	Submission	Set when a moderator promoted a submission

Field	Type	Notes
`recordStatus`	ready, hold, archived	Starts as hold

Each evidence item has a label, a kind, and one of a link, a document, a media file or a quote, plus the publisher and date. The server never fetches evidence links.

Whenever a published update is added, changed or removed, the system recalculates three fields on the claim. `currentStatus` becomes the status of the latest published update by date. `statusAsOf` becomes that update's date. `updateCount` becomes the number of published updates. A claim with no published update shows as "not yet assessed".

7.6 Topic

The data model calls this record type `categories`.

Field	Type	Notes
`slug`	Text	Unique
`name`	Text	Required. Localized
`description`	Text	Localized
`icon`, `color`		
`parent`	Topic	One level of nesting only
`order`	Number	

Topics are shared across countries.

7.7 Document

Field	Type	Notes
`country`	Country	Required
`title`	Text	Required
`actors`	Actors	Who the document is by or about
`type`	manifesto, speech, budget, policy, report, news, dataset, other	
`language`	`en` or `fr`	
`publishedOn`, `publisher`		
`file`	Media	The stored file
`sourceUrl`	Text	Shown as a link. Fetched only by the link connector, section 9.2
`connector`	Text	Which connector brought it in
`mimeType`	Text	The file type the system detected

Field	Type	Notes
`processing.status`	See section 9.4	The document's place in the pipeline
`processing.message`	Text	The last error or notice, shown in the admin
`processing.retryCount`	Number	How many times the pipeline has retried it
`text`	Text	The extracted text
`pageCount`	Number	
`recordStatus`	ready, hold, archived	Starts as ready

Documents are not public. Claim pages link to them as evidence, but only signed-in users can list them.

7.8 Submission

Field	Type	Notes
`claim`	Claim	Empty when someone reports a missing promise
`kind`	update, correction, new-claim	
`body`	Text	Required
`evidenceUrl`	Text	Shown as a link. Never fetched
`attachment`	Media	
`submitterName`, `submitterEmail`	Text	Optional
`consentToAttribute`	Yes or no	Starts as no
`status`	new, accepted, rejected, spam	Starts as new
`moderatorNote`, `moderatedBy`, `moderatedAt`		
`claimUpdate`	Claim update	Set when promoted
`ipHash`	Text	A scrambled form of the sender's address, for spotting abuse

Anyone can create a submission. Only signed-in users can read one, and the public site never shows one.

7.9 Extraction, owned by plugin-ai

An extraction is the record of one AI run over one document.

Field	Notes
`document`	Required

Field	Notes
`model`	Such as `google:gemini-2.5-pro`
`promptVersion`	So a change in results can be traced to a change in the prompt
`status`	pending, completed, failed, partial
`tokensUsed`, `durationMs`	
`candidates`	The list of candidates found

Candidate field	Notes
`summary`	The proposed title
`quotes`	Exact passages, each confirmed to appear in the document
`category`	The topic the AI suggested, as text
`confidence`	
`decision`	pending, accepted, rejected
`claim`	Set when accepted
`rejectedReason`, `reviewedBy`, `reviewedAt`	

Running AI on a document again creates a new extraction and keeps the old ones, so earlier decisions survive.

7.10 Supporting record types

Record type	Purpose
Media	Uploaded files. Each stores a checksum, so the same file is stored once
Users	Name, roles, and a country list that v3 does not enforce
Pages	Editorial pages. A page with no country is site-wide, such as `/about`. A page with a country belongs to it, such as `/ke/about`
Partners	Name, description, logo, link
Import jobs	One per spreadsheet import, holding the column mapping, the preview and the result
Workflow locks	Stop a scheduled job from running twice at once

Two settings records exist. **Site settings** hold the site title, logos, navigation and footer. **Settings** hold feature switches and each plugin's credentials. Only super-admins can read or change Settings, and the system encrypts every credential in it.

Credentials live in the database because they differ per deployment and change without a new release. Environment variables choose which plugins run. The database holds what they need to run.

7.11 Geo group

Countries and actors share one group of map fields.

Field	Notes
`centroid`	Latitude and longitude
`bounds`	North, south, east and west limits
`boundary`	An uploaded GeoJSON file with the outline
`boundarySource`, `boundaryLicense`	Required when an outline is present
`externalCode`	The outline's id in its source dataset

The system stores outlines as uploaded files and does not fetch them from a mapping service. Boundary datasets carry licence terms and rarely change, and a live dependency would break the map whenever the provider changed its terms. CONFIRM the step that simplifies large outlines before the public site serves them.

7.12 Languages

Content exists in English and French. Names, titles, summaries, narratives, status labels and topic names are localized. Slugs, codes, dates and the claim statement are not. A statement is an exact quote, and a translation of it would be a different statement.

7.13 Indexes

Record type	Indexes
Countries	`code` unique
Actors	Country, kind and slug together, unique. Country, kind and record status. Parent. Current office
Claims	Country and published status. Promisor and published status. Country and topic. Country and current status. Country and slug, unique. Verdict. Full-text on title, statement and summary
Claim updates	Claim and date, newest first
Documents	Processing status and last attempt time. Country
Extractions	Document. Candidate decision
Submissions	Status. Claim
Media	Checksum

8. Publishing rules

In short: an editor sets each record to ready, hold or archived. The system works out what is live. A record is live only when it is ready, everything above it is ready, and, for a claim, it is verified.

8.1 Two fields

`recordStatus` is what the editor wants.

Value	Meaning
ready	Fit to publish
hold	Not ready, or withdrawn for now
archived	Retired. Kept for audit and never shown

`_status` is what the system computed: `published` or `draft`. Nobody sets it directly.

```
A record is published when all of these are true:
  - its own recordStatus is ready
  - the recordStatus of every record above it is ready
  - for a claim, its verdict is verified
Otherwise it is a draft.
```

Record	Records above it that must be ready	Extra condition
Country	None	
Actor	Its country	
Document	Its country	
Claim	Its country, its promisor, and its source document if it has one	Verdict is verified
Claim update	Its country, promisor and claim	
Submission	Never published	

A claim's source document sits above it because the document is the evidence. If a manifesto turns out to be fake, setting it to hold removes every claim drawn from it.

Two fields are needed because one switch cannot say "this claim is fine, but its source is under review". With two, the editor's choice on each record survives. A claim that an editor held on its own stays held when its country is released. A claim that was hidden only because its country was held comes back.

8.2 The cascade

When an editor changes a record's `recordStatus`:

1. The system recomputes that record's `_status` before the save finishes.
2. A background job walks every record under it and recomputes each one.

3. When the job ends, the public site's cached pages for those records are refreshed, as described in section 16.3.

Because `_status` is always correct in the database, the public site asks one simple question of every list: is `_status` published? It never has to check parents.

A country with 5,000 claims and 20,000 updates is recomputed in one job that writes in batches. The job can be run again safely and picks up where it stopped if interrupted.

Changing a record's status never triggers AI extraction or verification again.

8.3 Deletion

The system never deletes a record as a side effect. When a record disappears from a connected system, the matching record here is archived. Only an editor or super-admin can delete, by hand, one record at a time. A document with reviewed candidates, verified claims and a published history represents work that cannot be redone.

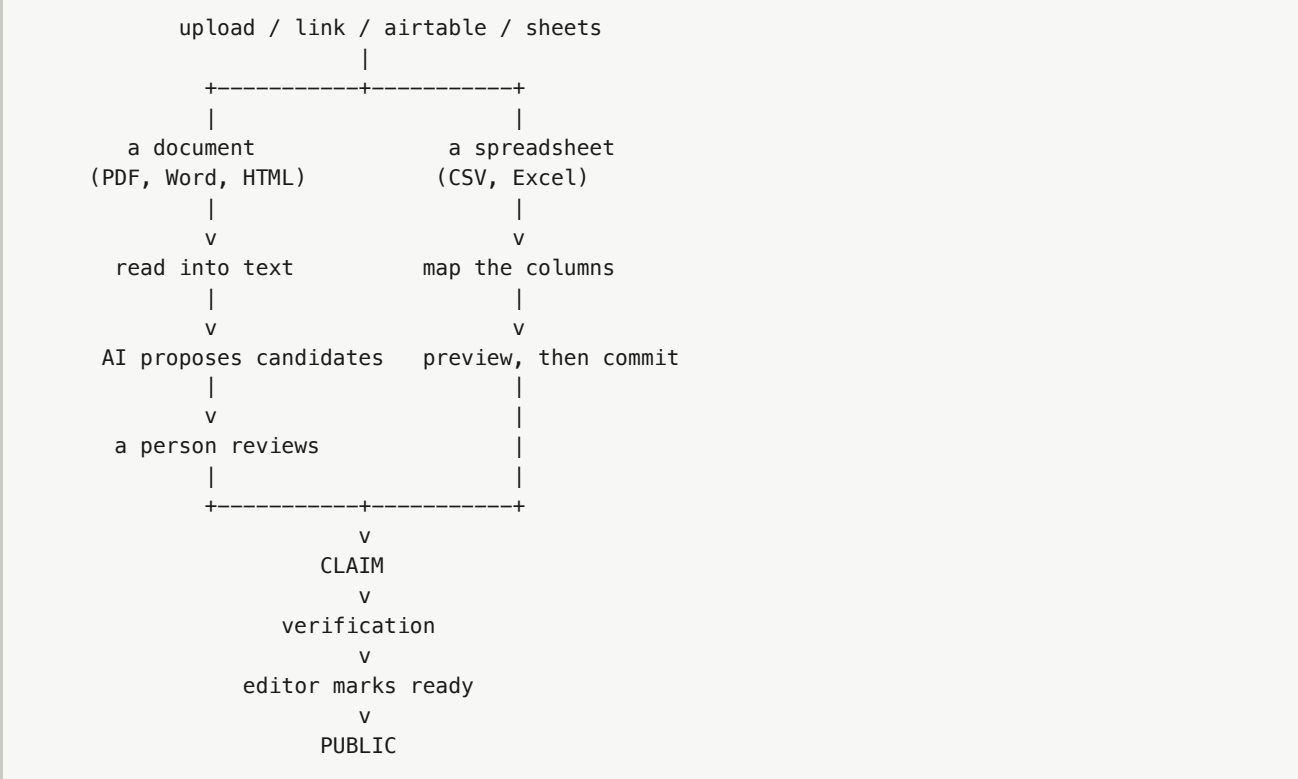
8.4 Audit trail

Event	Where it is recorded
A change of record status	Version history, and a log entry with the user and the old and new value
A change of verdict	Version history, with who verified it and when
A candidate accepted or rejected	On the candidate, with the reviewer, time and reason
A submission moderated	On the submission, with the moderator, time and note
A super-admin overriding a plugin's field	A warning in the log with the old and new value
A plugin sync	The plugin's own group on the record, with the time, result and any error

9. The pipeline

In short: content enters through a connector. Files are read into text, AI proposes candidates, and a reviewer accepts or rejects them. Spreadsheets skip those steps and become records directly. Either way the result is a claim, which is then verified and published.

9.1 The two paths



9.2 Connectors

Connector	In plugin	How it works
`upload`	core	An editor uploads a file in the admin
`url`	core	An editor pastes a link, and the server fetches the file through the safeguards in section 12.4
`airtable`	airtable	A scheduled job reads new rows from a base
`sheets`	sheets	Reads a Google Sheet as a spreadsheet source. Planned
`feed`	feed	Watches a publisher's RSS feed. Planned

The link connector is the only place where the server fetches an address a user typed. An editor asked for that address, the fetch passes every safeguard, and the system stores the file so it never fetches the address again.

9.3 Reading files

The system does not trust the file type a connector reports. It checks the first bytes of the file, then the type the web server declared, then the file extension. If these disagree, the document fails with a message saying why.

Extractor	In plugin	Reads
`plaintext`	core	Plain text, Markdown, and HTML with the page furniture removed
`tika`	tika	PDF, Word, Excel, PowerPoint, OpenDocument, RTF, EPUB and more
`ocr`	ocr	Scanned PDFs and images. Planned

An extractor returns clean text in a standard Unicode form, a page count if it knows one, and any title, author and date it found, which the editor can accept. It keeps every character. Version 2 strips all accented characters, which damages every French document.

A spreadsheet can be a document to read, such as a budget table, or a dataset whose rows are claims. The editor chooses at upload. CSV files default to the dataset path and Excel files to the document path.

9.4 Processing status

Each document carries one status that says where it is. This field is also the work queue.

```
pending > fetching > fetched > extracting > extracted
        > analysing > analysed > completed

Spreadsheet path:  pending > imported > completed
From any state:    failed, or needs-review
```

Every job writes the status before it starts and when it finishes. The admin list shows the status and any error message, so the person who uploaded a file can see what happened to it.

9.5 AI extraction and review

The method is the one v2 uses, which works well.

1. **First pass.** The system splits the text into overlapping sections and sends each to the selected model. The model returns candidates, each with a topic, a summary and exact quotes. The instructions ask for specific commitments about the future, forbid merging two commitments into one, and require quotes in the document's own language.
2. **Quote check.** The system removes duplicates, then discards every candidate whose quote does not appear word for word in the document. This check stops the AI from inventing claims. It must survive any future rewrite.
3. **Second pass.** The model splits compound commitments into single ones and writes titles. The quote check runs again.
4. **Save.** The result is one extraction record whose candidates all start as pending.

A reviewer then works through the candidates for a document. Each shows its title, topic, confidence, and its quotes highlighted in the source text. The reviewer accepts it, edits and accepts it, or rejects it with a

reason. Accepting creates a claim that is on hold and unverified.

Three parts of the v2 implementation are dropped: the fourteen-vendor settings screen, replaced by installing only the vendors in use. The per-document cost budget, replaced by a token limit and a time limit. And the separate export table, replaced by exporting straight from the claims.

9.6 Spreadsheet import

1. **Inspect.** The importer reads the column headings and the first rows.
2. **Map.** The editor matches columns to fields. The system saves the mapping for reuse.
3. **Preview.** The import runs without saving and reports how many rows it would create, update and skip, and which rows have errors.
4. **Commit.** The editor confirms, and the system saves.

An import can create claims, actors, claim updates and documents. Each has a natural key, such as country and slug for a claim, so importing the same file twice updates records instead of duplicating them. A row that names an actor or topic the system cannot find is reported as an error.

9.7 From claim to published

1. A claim starts on hold and unverified, however it was created.
2. The active verification plugin produces a verdict, as described in section 5.
3. An editor sets the claim to ready.
4. The claim goes live when the conditions in section 8.1 hold.
5. Editors add updates over time, and the latest published update sets the current status.

Verification and publishing are separate steps on purpose. A verified claim with an unfinished summary should not be live.

Claims with a verdict of false or misleading are never published and never deleted. They show where the AI or a source went wrong.

9.8 Handling failures

Retries

Error	Retries	Then
Network timeout	3, with growing waits	Move to the next item
Too many requests	Yes, after the wait the service asks for	Put the item back in the queue
Service unavailable	3	Skip. The next scheduled run retries
Invalid data	None	Mark the item failed with the reason
Sign-in or permission error	None	Fail and alert
Server error	2	Move to the next item

Waits start at one second, double each time, and stop growing at 30 seconds.

Time limits

Operation	Limit
Fetching a file	5 minutes
Reading a file into text	2 minutes
One AI request	2 minutes
All AI work on one document	20 minutes
One call to a verification service	1 minute

Stuck documents. A restart or a hung service can leave a document in a working state. A job runs every 5 minutes and looks for documents that have sat too long: 10 minutes in fetching, 5 in extracting, 25 in analysing. It restarts the stage and adds one to the retry count. After 5 retries it marks the document failed.

Pausing a failing service. After 5 failures in a row, the system stops calling a service for 60 seconds, then tries 3 test requests before resuming. Developers know this pattern as a circuit breaker.

Avoiding duplicates. Incoming webhooks carry an id or a body fingerprint, and the system ignores one it has already processed. Record creation relies on unique indexes in the database. Files are matched by checksum. Scheduled jobs take a lock, so a second run skips.

9.9 Background jobs

Job	In plugin	Runs when
`pullSource`	Each pull connector	On a schedule
`fetchDocument`	core	A document is created with a link to fetch
`extractText`	core	A document reaches fetched
`extractCandidates`	ai	A document reaches extracted
`runImport`	core	An editor commits an import
`submitForVerification`	The verification plugin	A claim is created
`pullVerdicts`	The verification plugin	Every 15 minutes
`recomputeClaimStatus`	core	A claim update changes
`cascadeStatus`	core	A record status changes
`sweepStuck`	core	Every 5 minutes
`cleanupJobs`	core	Hourly

10. The public site

In short: one site serves every country. Each page type has a clear web address. Pages are built once and reused until their content changes, so the site stays fast under heavy traffic.

10.1 Web addresses

```
/                              Home, with a country selector
/about, /methodology           Site-wide pages

/{cc}                          Country page
/{cc}/politicians              List of people
/{cc}/politicians/{slug}       A person's profile
/{cc}/parties                  List of parties
/{cc}/parties/{slug}           A party's profile
/{cc}/institutions             List of institutions
/{cc}/institutions/{slug}      An institution's profile
/{cc}/places                   List of places
/{cc}/places/{slug}            A place's profile
/{cc}/promises                 All claims, filtered and paged
/{cc}/promises/{slug}          A claim page
/{cc}/search                   Search results
/{cc}/{pageSlug}               A country's editorial page
```

`{cc}` is the country's two-letter code, such as `ke`, `za` or `ng`.

Each kind of actor has its own segment, so a slug only has to be unique within its country and kind. `/ke/politicians/nairobi` and `/ke/places/nairobi` can both exist. Editorial pages cannot use the reserved words: `politicians`, `parties`, `institutions`, `places`, `promises`, `search`, `api`, `embed`, `feed`.

The server works out the country once per request from a cached list and hands it to every part of the page. An unknown country code returns a "not found" page.

10.2 Pages

Claim page. In order:

1. Status, claim type, topic, the date the claim was made and its deadline, if it set one
2. The exact quote, with the promisor's name and a link to the source document and page
3. The editorial summary
4. The timeline, showing every published update with its date, status, account and evidence
5. The verdict, who verified it and when, and the rationale
6. A map, when the claim applies to a place that has map data
7. Share buttons and a "submit an update" form
8. Related claims by the same actor on the same topic

Actor profile. A scorecard comes first, showing the total number of claims and the share in each status group. A person's profile then shows their offices and party. The list of claims follows, with filters. A map shows where a person is from and where they govern.

Country page. A map of the country, headline numbers, featured actors, recently updated claims, and links into each list.

10.3 Lists and filters

- The server returns 24 claims per page.
- Filters cover status group, topic, actor, claim type and date range.
- The count beside each filter option comes from the database.
- The filter state is part of the web address, as in `?status=kept&category=health&page=2`, so a filtered view can be shared and bookmarked.
- Lists sort by latest update, deadline or status.

10.4 Search

- MongoDB full-text indexes cover claim titles, statements and summaries, and actor names and descriptions. Titles and names weigh more.
- Search covers one country by default, with an option to search all.
- Results are grouped into claims, people, parties, institutions and places.
- Results have a web address, `/{cc}/search?q=`, so they can be linked.
- All search code sits in one module, so a move to a dedicated search engine changes one place.

CONFIRM the volume at which MongoDB full-text search stops being adequate.

10.5 Being found

Feature	How
Sitemaps	An index at `/sitemap.xml` with one sitemap per country
`robots.txt`	Generated, and points to the sitemap index
Canonical addresses	Every page declares its one official address
Share images	Generated for each claim and actor
Fact-check markup	Every verified claim page carries ClaimReview data
Profile markup	Actor pages carry Person, Organization or Place data
Feeds	Each country has an RSS feed of new and updated claims

ClaimReview is the standard format for fact-checks from schema.org. Search engines read it to show fact-check results, and other fact-checking tools exchange it.

```
{
  "@context": "https://schema.org",
  "@type": "ClaimReview",
  "url": "https://promisetracker.org/ke/promises/...",
  "claimReviewed": "We will build 200 new health centres by 2027",
  "itemReviewed": {
    "@type": "Claim",
    "author": { "@type": "Person", "name": "William Ruto" },
    "datePublished": "2022-06-30"
  },
  "author": { "@type": "Organization", "name": "PromiseTracker" },
  "reviewRating": {
    "@type": "Rating",
    "ratingValue": 2,
    "bestRating": 5,
    "worstRating": 1,
    "alternateName": "In progress"
  }
}
```

The rating number comes from the status group, and `alternateName` is the status label.

10.6 Maps

A map draws from the geo group on the country, the place, or the place a promisor governs. It shows the outline if one exists. If not, it shows a marker at the centre point. If there is no map data, the page shows no map.

Maps load after the rest of the page and below the first screen, so they never slow the page down. CONFIRM the map tile provider and its licence for commercial and embedded use, and the source of boundary data and its attribution rules.

10.7 Languages and accessibility

Each country publishes in English, French or both. The site has a visible language switch, remembers the choice, and tells search engines about each language version of a page. Every label in the interface is translated.

Requirement	Standard
Colour contrast	WCAG AA, including every status colour. The admin warns an editor who picks a status colour that fails
Keyboard	Every filter, timeline control and map control works without a mouse
Screen readers	The timeline is a real ordered list

11. APIs

In short: the public can read published data through a documented API and embed cards in other sites. Connected systems reach the plugins through a small set of protected routes.

11.1 Public read API

```
GET /api/public/v1/countries
GET /api/public/v1/{cc}/actors?kind=person&page=1
GET /api/public/v1/{cc}/claims?status=kept&category=health
GET /api/public/v1/{cc}/claims/{slug}
GET /api/public/v1/{cc}/claims/{slug}/updates
```

- No sign-in. Requests are limited per address and cached.
- It returns published records only, using the same rule as the site.
- Its response format is documented and stable. It is separate from the internal format, so internal changes do not break people who use it.
- A machine-readable description is at `/api/public/v1/openapi.json`.
- A setting switches it on and off.

Claims and their updates can also be exported as CSV, using the same filters.

11.2 Embeds

```
GET /embed/claim/{id}            One claim, with status and last update
GET /embed/scorecard/{actorId}   An actor's status breakdown
GET /embed/list?...              A filtered list of claims
GET /api/oembed?url=...          Lets a CMS turn a pasted link into a card
```

Embeds are the only pages other sites may place in a frame. They set no cookies, need no sign-in and track nothing. Each links back to the full page. A setting switches them on and off.

11.3 Core routes

Route	Purpose
`GET /api/health`	Full report on every enabled service and plugin
`GET /api/health/ready`	Database and file storage only. Used to decide whether to send traffic
`POST /api/submissions`	Creates a citizen submission. Rate limited, with a captcha
`GET /sitemap.xml`	Sitemap index
`GET /{cc}/feed.xml`	A country's RSS feed

11.4 Plugin routes

Each route returns `503` when its plugin is disabled.

Route	Protection	Purpose
`POST /api/plugins/superdesk/items`	Signature over the body	Receives published Superdesk items
`GET, POST /api/plugins/superdesk/assets`	Signature over the body	Media for those items
`POST /api/plugins/check/webhook`	Shared secret in the `Authorization` header	Receives Check report events
`POST /api/plugins/airtable/webhook`	Shared secret	Receives Airtable change notices

Each handler returns `401` for a bad signature or secret without reading the body. It returns `200` for an item it does not recognise, so the sender stops retrying, and leaves the item for the next poll.

12. Security

In short: the public site needs no sign-in and exposes published data only. Everything else is behind roles. The server does not fetch addresses that users type, files are checked before they are stored, and credentials are encrypted.

12.1 Who can reach what

Area	Protection
Public site, public API, embeds	None
Admin	Signed-in session
Plugin routes	A shared secret or signature per route

12.2 Roles

A user can hold more than one role.

Role	Purpose
super-admin	Full access. Manages users, settings, credentials, statuses and topics
editor	Decides what is public. Controls record status everywhere and owns the claim timeline
contributor	Adds actors, documents and draft claims. Cannot publish
reviewer	Reviews AI candidates and sets verdicts. Loses the verdict when a verification plugin is enabled
moderator	Handles submissions and can turn one into a claim update
viewer	Read-only

Action	super-admin	editor	contributor	reviewer	moderator
Create or edit countries	Yes	Yes			
Create or edit actors and documents	Yes	Yes	Yes		
Create claims by hand	Yes	Yes	Yes		
Accept or reject AI candidates	Yes	Yes		Yes	
Set a verdict	Yes	Yes		Yes	
Create or edit claim updates	Yes	Yes			From a submission
Set record status	Any	Any	Hold only	On claims	
Moderate submissions	Yes	Yes			Yes
Delete a record	Yes	Yes			
Override a plugin's field	Yes, logged				
Manage users and settings	Yes				

Every role can read every record. A viewer can do nothing else.

Version 3 does not limit editing by country. Any user with a write role can edit any country's records. Adding the limit later needs a filter in the access rules and no change to stored data, because every record already carries its country. It must be in place before an outside newsroom gets an account.

12.3 Checking input

The system checks all outside data against a defined shape before using it. That covers webhooks, API responses, spreadsheet rows and form submissions.

A webhook handler works in a fixed order:

1. Check the signature or secret, before reading the body.
2. Check whether this delivery was already processed.
3. Check the shape of the content.
4. Only then write to the database.

12.4 Fetching outside addresses

A server that fetches any address a user types can be tricked into reaching private systems. This attack is called server-side request forgery, or SSRF.

By default the server fetches nothing a user typed. Source links, evidence links and submission links are stored and shown, and never fetched.

The link connector is the one exception, and it applies these safeguards, all taken from v2:

- HTTPS only.
- Before connecting, the server looks up the address and refuses any private, local or reserved network address, including cloud metadata addresses.
- The server follows redirects itself, checks each hop the same way, and limits the number of hops.
- Time limits apply to each hop and to the whole fetch.
- A size limit applies, checked from the declared size and again while downloading.
- The file type must match the file's first bytes. SVG files are refused.

One small risk remains. A hostile server could change its address in the moment between the check and the connection. The hosting environment's outbound network rules cover that case.

12.5 Uploads

- The file extension, the declared type and the file's first bytes must all agree.
- Each kind of file has a size limit.
- Each client has a rate limit and a daily volume limit.
- The system stores each file by its checksum, so identical uploads are stored once.

12.6 Cleaning content

The system strips extracted text and outside HTML down to a short list of safe tags before storing it. It never stores a visitor's network address. Submissions keep a scrambled form of the address, which is enough to spot repeated abuse and cannot identify a person.

12.7 Credentials

Credential	Where it lives	Replaced
Database and application secrets	Environment	Yearly
File storage keys	Environment	Yearly
Plugin API keys and tokens	Settings, encrypted	When a staff member with access leaves
Webhook secrets	Environment	Every 6 months

The encrypted settings field comes from v2. It encrypts on save, shows a mask in the admin, and keeps the stored value when someone saves the form without retyping it.

The system never writes keys, tokens, passwords, submitter email addresses or network addresses to its logs.

12.8 Browser security policy

The site tells browsers that no other site may place its pages in a frame. Embed routes are the one exception. This keeps the admin and the main site from being wrapped inside a hostile page, while partners can still embed cards.

12.9 Protecting public routes

Route	Protection
Submission form	Captcha, rate limit per address, a hidden trap field for bots
Public API	Rate limit per address, caching, a page size limit
Search	A limit on query length, rate limit
Plugin routes	Secret or signature, rate limit, duplicate detection

12.10 How long data is kept

Data	Kept for
Claims, updates, actors, documents	Indefinitely. Archived, never removed automatically
Raw AI responses	90 days. Candidates and decisions are kept
Rejected and spam submissions	180 days
Submitter contact details	Until the submitter asks for removal. Never published without consent
Failed background jobs	7 days

13. Errors and logging

In short: every error has a code, says which record it concerns, and says whether retrying can help. Problems an editor can fix appear in the admin. Problems only a developer can fix go to the error tracker.

13.1 Error types

Error type	Raised when	Retried
ConfigurationError	A setting is missing or wrong at start-up	No. The system does not start
PortResolutionError	A port has no adapter, or more than one	No. The system does not start
ExternalServiceError	An outside service fails or times out	Depends on the response
ExtractionError	Reading a file or running AI fails	Yes, within the retry budget
ValidationError	Incoming data has the wrong shape	No
OwnershipError	Something tries to write a field a plugin owns	No

Each error carries a stable code, the ids of the records involved, and a flag saying whether a retry can help. Retry logic reads that flag.

13.2 Logs

Every log entry is structured and carries the level, message, time, source, and the ids needed to follow one document through the pipeline: document, claim, country, job and plugin. Slow operations record how long they took.

The system logs successes as well as failures, because a drop in successes is often the first sign of trouble. Entries go to the console and to Sentry.

13.3 Where a problem shows up

Problem	Where it appears	Who fixes it
A document cannot be read	On the document, with the reason. The uploader gets an email	The uploader replaces the file or link
AI finds no usable candidates	The document is marked needs-review	An editor reads it and enters claims by hand
A verification service is down	Health check shows degraded	No one. The poll catches up when it returns
A spreadsheet row names an unknown actor	In the import preview, by row number	The editor fixes the sheet or adds the actor
The start-up configuration is wrong	The system refuses to start and lists every problem	A developer

13.4 Notifications to staff

Event	Email sent
A record is archived, or brought back from the archive	Yes
A country is set to hold	Yes. It hides everything under it and is easy to do by accident
An actor or document is set to hold	No. It is routine
A document fails after all retries	Yes, to the uploader
New submissions arrive	A daily digest to moderators

14. Testing

Kind	Share of tests	Covers
Unit	About 80%	Single functions: the publishing rule, slugs, the quote check, column mapping, retry timing
Integration	About 15%	Jobs with outside services simulated, routes, access rules, the cascade, plugin loading
End to end	About 5%	A document from upload to public page. A submission from form to timeline

Coverage targets are 70% overall, 85% for the pipeline, publishing rules and plugin loading, and 90% for utilities.

Two rules are specific to this design.

1. Each plugin is tested switched on and switched off. Switched off, it must leave the configuration exactly as it found it.
2. The core is tested alone, with no credentials for any outside service. If those tests need an API key, the first rule in section 1.5 is broken and the build fails.

The security tests from v2 carry over unchanged and must keep passing.

15. Deployment and operations

15.1 Start-up checks

Before it serves any request, the system checks that:

1. The core environment variables are present.
2. The plugin list loads, with every dependency met.
3. Each port that needs one adapter has exactly one.
4. Each enabled plugin has its required variables.
5. The database and file storage respond.
6. Each enabled plugin's own settings are valid. The Superdesk plugin checks its ids against the live Superdesk, and the Check plugin checks its status mapping against the live workspace.

If anything fails, the system prints every problem and exits.

15.2 How it is deployed

- One application container. Tika runs in a second container when its plugin is enabled.
- The database and file storage are managed services outside the container.
- New versions roll out without downtime. A new instance receives traffic only after its readiness check passes.
- A release is rolled back if the error rate stays above 10% for 5 minutes, if a main function breaks, or if data is damaged.

15.3 Health checks

`/api/health` reports the enabled plugins, the adapter chosen for each port, and the state and response time of each service. A plugin that is not enabled does not appear.

```
{
  "status": "healthy",
  "plugins": {
    "enabled": ["core", "tika", "ai", "check"],
    "ports": { "verification": "check" }
  },
  "services": {
    "database": { "healthy": true, "latencyMs": 4 },
    "storage":  { "healthy": true, "latencyMs": 22 },
    "tika":     { "healthy": true, "latencyMs": 31 },
    "llm":      { "healthy": true, "latencyMs": 180 },
    "check":    { "healthy": true, "latencyMs": 95 }
  }
}
```

`/api/health/ready` checks the database and file storage only. If Tika, the AI provider or the verification service is down, the pipeline slows or pauses, and the public site keeps serving.

15.4 Alerts

Condition	Goes to
Pipeline success rate below 90% for 15 minutes	Team chat
A service unhealthy for more than 5 minutes	On-call
Error rate above 5% for 10 minutes	Team chat
A rise in rejected webhooks	Team chat
Three empty verification polls in a row	Team chat
AI account out of credit	Team chat

15.5 Backups

The database is backed up in full every day and backups are kept for 30 days. File storage keeps earlier versions of each file and copies them to a second region. The targets are recovery within 4 hours, losing at most 24 hours of data. The team tests a restore every quarter.

15.6 Setting up a new deployment

A new deployment needs a few records before any claim can exist. They must be created in this order.

Order	Record	Why it comes first	Minimum
1	Claim statuses	Every update needs one, and every scorecard reads its group	A full list, each with a group
2	Topics	Every claim needs one	8 to 15
3	Countries	Web addresses start with the country	One
4	Users	Roles control every later step	One super-admin and one editor
5	Site settings	Navigation and footer	Enough to show a page
6	Actors	Every claim needs a promisor	The actors being tracked

The status groups deserve care. Nothing supplies a default, and a status in the wrong group makes every scorecard wrong without any error appearing.

16. Performance

16.1 Speed targets

Operation	Target
A cached country or claim page	Under 100 ms at the server
Public API	Under 300 ms for 95% of requests
Health check	Under 100 ms
Main content visible on a 4G phone	Under 2.5 seconds
Page ready to use	Under 3 seconds
Reading a file into text	Under 2 minutes
AI work on one document	Under 3 minutes typically
A document through the whole pipeline	Under 15 minutes for 95% of documents

A claim page sends less than 150 KB of compressed JavaScript, not counting the map.

16.2 Growth targets

	Phase 1	Phase 2	Phase 3 and later
Countries	5	20	50 or more
Actors	200	2,000	10,000
Claims	2,000	25,000	150,000
Claim updates	5,000	75,000	500,000
Documents	500	10,000	50,000
Visitors per month	20,000	200,000	1,000,000

Kind of growth	What handles it
More countries	A country is one record. Adding one needs no deployment
Traffic spikes	Cached pages, pages built ahead of time for published records, and a public site with no outside dependency
More claims	Paging and filtering in the database, an index behind every list, and a cascade that writes in batches
More staff	One work queue, a review queue with a decision per candidate, record locking in the admin, and separate roles for adding and publishing

16.3 Caching

The site builds a page once and serves the stored copy until the content behind it changes. Each stored page carries tags, and a change refreshes the pages with the matching tag.

Tag	Refreshed when
`country:{code}`	The country changes
`actor:{id}`	The actor, or a claim naming it, changes
`claim:{id}`	The claim or one of its updates changes
`claims:{country}`	A claim in that country is published or withdrawn
`statuses` , `categories`	The list changes
`settings`	Site settings change

Pages for published records are built ahead of time. Less-visited pages are built on first visit and then stored. The cascade job in section 8.2 triggers the refresh, so a change is live within seconds. Media files are cached by browsers for a year, since a changed file gets a new address.

17. Development workflow and delivery

17.1 Working on the code

`main` is the production branch. Work happens on a branch and merges through a pull request. Commit messages follow the Conventional Commits format.

Task	Command
Install	`pnpm i`
Run locally	`pnpm dev`
Build	`pnpm build`
Check code style	`pnpm lint`
Integration tests	`pnpm test:int`
End-to-end tests	`pnpm test:e2e`
Regenerate types	`pnpm generate:all`
Start Tika locally	`docker compose up -d`

Automated checks run the linter and both test suites with only the core plugin enabled and no credentials.

17.2 Delivery phases

Phase	What is built	Done when
1. Core	Kernel and plugin loading. The data model. Publishing rules and roles. Upload, link and spreadsheet import. Plain text reading. Built-in verification. The public site with caching, lists, search and fact-check markup. Submissions and moderation	A non-developer sets up a country, enters an actor and a claim, adds three dated updates, publishes, and sees the timeline on the public page, with no API key configured
2. Scale and assist	Editing rights per country. plugin-tika. plugin-ai with one vendor and the review queue. Maps. Embeds, the public API and CSV export. The language switch	A 200-page PDF produces reviewed claims, and an editor limited to one country cannot edit another
3. Newsroom	plugin-superdesk and plugin-check, each with its push route, poll and start-up checks	A verdict published in Superdesk or Check appears on the claim, and a withdrawn one takes the claim down

Editing rights per country come first in phase 2, because an outside newsroom cannot safely have an account without them.

18. Design decisions

Why is the core a plugin? So that it follows the same extension rules as every integration. If the rules are too weak for Superdesk, the core feels it too, and the gap is found early. It also keeps integrations out of the core's internals.

Why must the system run with no outside service? A system that needs two outside services cannot be tested automatically, cannot be handed to a new developer without accounts, and stops when either vendor changes its terms. Testing the stand-alone configuration on every change keeps it working.

Why refuse to start when a port has no adapter? A missing optional feature can fail quietly. A missing required one cannot. A deployment with nowhere to verify claims is broken, and the person deploying it should be the one who finds out.

Why are a plugin's fields read-only in the admin? If two systems can write one field, they will disagree. Making the field read-only keeps the value visible while leaving one writer.

Why are a plugin's added fields optional? A plugin can be removed. Optional fields let the remaining records stay valid.

Why choose plugins with environment variables? Plugins change the shape of records and the list of routes, and both are fixed when the system starts. A database setting would be read too late.

Why keep credentials in the database? They differ per deployment and change without a release. The encrypted settings field already protects them.

Why both a push and a poll for verdicts? A push is fast and can be lost. A poll is slow and cannot. Together they give a quick result in the normal case and a correct one in every case.

Why a country in the address and not a separate site? A separate site per country means a separate address, certificate and cache for each, and makes cross-country pages impossible. One site with the country in the path avoids all of that and still labels every record.

Why a segment per kind of actor? Without it, a slug would have to be unique across people, parties, institutions and places, and the server would have to search all four to find a page.

Why one record type for all actors? They all make promises. Four record types would mean four sets of pages and a combined query for every list.

Why derive a claim's current status? A stored status can drift from the evidence. A derived one always matches the latest update, its date and its sources.

Why two publishing fields? One switch cannot hold both "the editor wants this public" and "its parent is withdrawn". Two fields keep the editor's choice intact while the parent is hidden.

Why does a source document sit above its claims? The document is the evidence. Withdrawing a fake manifesto should take one action.

Why check quotes against the source? An AI model can produce a convincing promise that nobody made. One text search removes that risk.

Why three ingestion ports? A spreadsheet of promises and a manifesto need different handling. One needs column mapping, the other needs reading, AI and review.

Why keep the claim in PromiseTracker when verifying elsewhere? Version 2 kept promises in Check and rebuilt them from its reports, so any change in Check could remove a promise. Sending only a copy out and taking only the verdict back means no outside system can do that.

Why store map outlines? They rarely change, they carry licence terms, and a live map-data service is one more thing that can change under us.

Why Payload, MongoDB and Tika? Payload supplies the admin, typed records, background jobs and a plugin system that supports the ownership rules. MongoDB suits a model that is still changing and is the database Payload supports best. Tika is free, self-hosted and reads most formats.

19. Appendix A: items to confirm

Superdesk

1. The Content API address prefix, and whether the Content API is switched on. Section 5.2
2. Whether the Production API is running. Section 5.2
3. The content profile id for fact-check items, and whether one exists yet. Section 5.2
4. The vocabulary ids and codes for verdict, claim type and topic. Section 5.2
5. The desk and stage ids for the fact-checking desk. Section 5.2
6. A service account with the `archive` privilege, agreed with the newsroom. Section 5.2
7. Whether HTTP push is available, and whether PromiseTracker can be added as a subscriber. Section 5.2
8. The Superdesk version in use. Its published change log stops in 2022. Section 5.2
9. Whether `extra` fields are searchable on this installation. Section 5.2

Meedan Check

10. The workspace id, and an API key with permission to create items and read reports. Section 5.3
11. The workspace's status list, and the verdict each status maps to. Section 5.3
12. Whether the workspace can send webhooks to an outside address, and which events it sends. Section 5.3
13. The report state names, `published` and `paused`, in the current Check version. Section 5.3

Statuses and topics

14. The verdict list. The current four answer "is this true". A promise tracker more often asks "did they say it" and "did they do it". Decide before setting up Superdesk vocabularies or the Check mapping. Section 7.3
15. The claim status list and the group for each status. Section 15.6
16. The topic list. Section 15.6
17. Whether the four claim types are right, and whether any needs its own fields. Section 7.3

Maps and search

18. The map tile provider and its licence. Section 10.6
19. The boundary data source and its attribution rules. Section 10.6
20. The step that simplifies large outlines. Section 7.11
21. The volume at which MongoDB full-text search stops being adequate. Section 10.4

20. Appendix B: what v2 does today

This appendix records the v2 behaviour that v3 replaces, with the place in the v2 code where each can be seen, and the v2 code that v3 keeps.

Problems v3 fixes

Problem	Where to see it in v2
Airtable is the source of documents and the work queue	Six pipeline tasks write status text into Airtable cells
Only the Check sync creates promises	`promises.meedanId` is required and unique in `src/collections/Promises.ts`
No promise history	The timeline component receives an empty list of events
Nothing is cached	Every public route sets `force-dynamic`
The promise list loads everything	`pagination: false` in `src/lib/data/promiseQueries.ts`
Search does nothing	`src/components/Search.tsx` has no handler
Three publishing conventions	Two yes-or-no fields and one free-text field, `promises.publishStatus`
Tenant isolation is not enforced	`userHasAccessToAllTenants: () => true` in `src/plugins/index.ts`
Accented characters are removed	`cleanText` in `src/tasks/extractDocuments.ts`
Pages cannot be embedded	`frame-ancestors 'self'` in `src/lib/security/headers.mjs`, with no exception

Code v3 keeps

What	Where in v2
Safeguards for fetching outside addresses	`src/utils/ssrf.ts`, `src/utils/files.ts`
File type checking	`src/utils/fileSignature.ts`
The AI quote check	`src/tasks/extractPromises.ts`
The AI vendor list	`src/lib/ai/providerCatalog.ts`, `src/lib/ai/providerRegistry.ts`
The Check API client, as the base for plugin-check	`src/lib/meedan.ts`
Webhook duplicate detection	`src/app/api/meedan-sync/route.ts`
The encrypted settings field	`src/fields/encryptedSecret.ts`
The scheduled job lock	`src/lib/workflowLocks.ts`
The security tests	`tests/int/`